```
>>>conn.close()  //closes the connection
>>>conn.callproc(proc,param)  //call stored procedure for execution
>>>conn.getsource(proc)  //fetches stored procedure code
```

Cursor objects

Cursor is one of the powerful features of SQL. These are objects that are responsible for submitting various SQL statements to a database server. There are several cursor classes in *MySQLdb.cursors*:

1. *BaseCursor* is the base class for Cursor objects.
2. *Cursor* is the default cursor class. *CursorWarningMixIn*, *CursorStoreResultMixIn*, *CursorTupleRowsMixIn*, and *BaseCursor* are some components of the cursor class.
3. *CursorStoreResultMixIn* uses the *mysql_store_result()* function to retrieve result sets from the executed query. These result sets are stored at the client side.
4. *CursorUseResultMixIn* uses the *mysql_use_result()* function to retrieve result sets from the executed query. These result sets are stored at the server side.

The following example illustrates the execution of SQL commands using cursor objects. You can use *execute* to execute SQL commands like *SELECT*. To commit all SQL operations you need to close the cursor as *cursor.close()*.

```
>>>cursor.execute("SELECT * FROM books")
>>>cursor.execute("SELECT * FROM books WHERE book_name = 'python' AND
book_author = 'Mark Lutz' ")
>>>cursor.close()
```

Error and exception handling in DB-API

Exception handling is very easy in the Python DB-API module. We can place warnings and error handling messages in the programs. Python DB-API has various options to handle this, like *Warning, InterfaceError, DatabaseError, IntegrityError, InternalError, NotSupportedError, OperationalError* and *ProgrammingError*.

Let's take a look at them one by one:

1. *IntegrityError*: Let's look at integrity error in detail. In the following example, we will try to enter duplicate records in the database. It will show an integrity error, *_mysql_exceptions.IntegrityError*, as shown below:

```
>>> cursor.execute('insert books values (%s,%s,%s,%s)',('Py9098','Programmin
g With Perl',120,100))
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 95, in
execute
   return self._execute(query, args)
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 114, in
_execute
   self.errorhandler(self, exc, value)
   raise errorclass, errorvalue
_mysql_exceptions.IntegrityError: (1062, "Duplicate entry 'Py9098' for key 1")
```

2. *OperationalError*: If there are any operation errors like no databases selected, Python DB-API will handle this error as *OperationalError*, shown below:

```
>>> cursor.execute('Create database Library')
>>> q='select name from books where cost>=%s order by name'
>>>cursor.execute(q,[50])
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 95, in
execute
   return self._execute(query, args)
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 114, in
_execute
   self.errorhandler(self, exc, value)
  File "/usr/lib/python2.3/site-packages/MySQLdb/connections.py", line 33, in
defaulterrorhandler
   raise errorclass, errorvalue
_mysql_exceptions.OperationalError: (1046, 'No Database Selected')
```

3. *ProgrammingError*: If there are any programming errors like duplicate database creations, Python DB-API will handle this error as *ProgrammingError*, shown below:

```
>>> cursor.execute('Create database Library')
Traceback (most recent call last):>>> cursor.execute('Create database Library')
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 95, in
execute
   return self._execute(query, args)
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 114, in
_execute
   self.errorhandler(self, exc, value)
  File "/usr/lib/python2.3/site-packages/MySQLdb/connections.py", line 33, in
defaulterrorhandler
   raise errorclass, errorvalue
_mysql_exceptions.ProgrammingError: (1007, "Can't create database 'Library'.
Database exists")
```

Python and MySQL

Python and MySQL are a good combination to develop database applications. After starting the MySQL service on Linux, you need to acquire MySQLdb, a Python DB-API for MySQL to perform database operations. You can check whether the MySQLdb module is installed in your system with the following command:

```
>>>import MySQLdb
```

If this command runs successfully, you can now start writing scripts for your database.

To write database applications in Python, there are five steps to follow:

1. Import the SQL interface with the following command:
   `>>> import MySQLdb`
2. Establish a connection with the database with the following command:
   `>>> conn=MySQLdb.connect(host='localhost',user='root',passwd='')`
   ...where *host* is the name of your host machine, followed by the user name and password. In case of the root, there is no need to provide a password.